

PROMPTVER: Semantic Versioning for Prompt Edits

Author Name(s) Omitted for Review

Institution Omitted for Review

Email Omitted for Review

Abstract—Software built on large language models (LLMs) often depends on prompts that change over time. Even small prompt edits can affect the software that uses them. A reliable way to assess these edits would be to execute the old and new prompts on many inputs and models. However, doing this after every prompt change is costly and time-consuming during development. Developers therefore need an earlier signal that estimates whether a prompt edit is safe, backward-compatible, or breaking.

We propose PROMPTVER, a structured approach that labels prompt edits using semantic versioning convention: PATCH for behavior-preserving changes, MINOR for backward-compatible extensions, and MAJOR for breaking changes. Given an original prompt and an edited prompt, PROMPTVER decomposes each prompt into five components: role, directive, constraints, examples, and output format. It then compares these components, checks for possible contract changes, and assigns a severity label. At classification time, PROMPTVER does not execute the prompt; the label is derived from the edit itself.

We evaluate PROMPTVER on 141 prompt edits across classification and summarization tasks. Reference labels are built by executing the original and edited prompts and checking their behavioral and contract-level differences. On classification tasks, PROMPTVER improves accuracy over an LLM-only baseline by 8.2 percentage points and improves MAJOR recall from 79.5% to 91.8%. On summarization tasks, the improvement is smaller, showing that open-ended generation needs better calibration. Beyond the final label, PROMPTVER produces a structured decision trace that shows which component changed and which rule triggered the label.

Index Terms—prompt engineering, semantic versioning, LLM applications, prompt evolution, software maintenance.

I. INTRODUCTION

Large language models (LLMs) are now used in many types of software, including chatbots, search tools, code assistants, and data-extraction pipelines [21], [23]. In these systems, prompts play a central role. They define the task, the rules to follow, and the expected output format. When a developer edits a prompt, the edit can therefore change how the software behaves.

Recent studies show that developers frequently edit prompts in real software projects [6], [21]. They add instructions, remove constraints, change examples, and rewrite prompt text. These edits may look small, but their effect can be large. A format change can break a parser. A new input variable can break the calling code. A removed constraint can change behavior expected by another component. Today, developers

have no standard way to assess the impact of such changes before deployment.

This problem is difficult for three reasons. First, prompts are written in natural language, so they have no compiler, type system, or formal interface. A text diff cannot reliably distinguish a harmless rewording from a behavior-changing edit [15], [19]. Second, the effect of a prompt edit depends on the model, the input data, and the downstream code that consumes the output [5], [19]. Third, prompts have no explicit contract. Unlike a function signature, a prompt does not clearly declare what it accepts and what it returns. Its contract is implicit in the prompt text, the model, the input format, and the application code [18].

A fully reliable severity assessment would require testing the edited prompt with many models, inputs, and downstream uses. This is often too costly during development. Still, developers need an early signal before a prompt edit is merged into production.

In conventional software, semantic versioning provides a simple release-impact signal [12]. PATCH means a safe fix, MINOR means a backward-compatible addition, and MAJOR means a breaking change. This convention helps developers communicate whether an upgrade is safe [3], [9], [14]. Prompts do not yet have an equivalent convention. A prompt edit is usually reviewed as plain text, with no clear label saying whether it is safe, compatible, or breaking.

We propose PROMPTVER, a framework for labeling prompt edits using semantic versioning. Given an original prompt and an edited prompt, PROMPTVER decomposes each prompt into components, compares what changed, checks whether the edit may affect the prompt contract or the software that depends on it, and assigns one of three labels: PATCH, MINOR, or MAJOR.

We evaluate PROMPTVER by executing original and edited prompts and measuring whether their behavior changes using flip rate for classification tasks and semantic drift via BERTScore for summarization tasks. The evaluation uses 141 prompt-edit pairs and produces 436 valid edit-executor runs across classification and summarization tasks. Reference labels (ground-truth severity annotations for each prompt edit) combine observed behavior with explicit contract checks, such as changed input variables, output formats. PROMPTVER reaches 71.8% agreement with these reference labels. A direct LLM baseline is competitive, especially on summarization, which suggests that PROMPTVER’s value is not only the final

label but also the decision trace that explains why the label was assigned.

This paper makes three contributions:

- 1) We introduce PROMPTVER, a semantic versioning framework that classifies prompt edits as PATCH, MINOR, or MAJOR using prompt decomposition, edit operators, and contract checks.
- 2) We build a controlled benchmark of 141 prompt-edit pairs, with reference labels derived from prompt execution and explicit contract-violation checks.
- 3) We compare it with two baselines, showing stronger MAJOR recall on classification tasks and a structured decision trace for reviewing prompt-edit severity.

PROMPTVER removes the need for costly execution-based testing after every prompt edit, giving developers an early, automated signal that saves review time and prevents silent regressions from reaching production.

II. THE PROMPTVER FRAMEWORK

PROMPTVER takes two prompt versions as input: the original prompt and the edited prompt. It outputs one SemVer label: PATCH, MINOR, or MAJOR. It also outputs an explanation of why this label was assigned. The following subsections describe how PROMPTVER decomposes prompts, defines edit operators, and applies a rule-guided procedure to assign this label.

A. Versioning Unit and Template Decomposition

A deployable prompt artifact is treated as a configuration $U = \{P, H, M, C, T\}$, where P is the prompt text; H is the hyperparameter set (temperature, top- k , seed); M is the model snapshot (a pinned identifier such as `gpt-4-0613`, never `latest`); C is the context (input e.g. RAG); and T is the validation test set. *In this paper our experiments vary only P while pinning M ; H , C , and T are fixed by the benchmark configuration.* We retain them in the framework definition because production prompt artifacts are versioned jointly with these elements (e.g., a temperature change with no P edit can still alter consumer-visible behavior), but their empirical evaluation is left to future work.

The template P is further decomposed:

$$P = \langle R, D, Ctx, E, O \rangle, \quad (1)$$

where R is an optional role/persona; D is the core directive; Ctx is a list of contextual constraint sentences; E is a list of examples (empty for zero-shot); and O is the output-schema specification. This decomposition exposes the *contract surface*: D and O define what the prompt promises to its caller, while R , Ctx , and E define how the model is steered. SemVer rules distinguish edits that alter the contract surface (typically MAJOR) from edits that adjust steering (typically MINOR or PATCH).

The decomposition synthesizes prior work. Tafreshpour et al. [21] empirically identified recurring components in 1,262 prompt commits across 243 repositories. Morales et al.’s Impromptu [8] formalized prompts as compositions of traits

TABLE I
PROMPTVER OPERATOR FAMILIES. THE FAMILY IDENTIFIES THE CHANGED COMPONENT; THE CLASSIFICATION PROCEDURE, NOT THE FAMILY ALONE, DETERMINES SEVERITY.

Family	IDs	Comp.	Typical transformations
Instruction	E_1-E_3, E_{15}	D	Add, remove, rewrite, or restructure task directives.
Constraint	E_4-E_6	Ctx	Add, remove, or modify behavioral or validity constraints.
Example	E_7-E_9, E_{16}	E	Add, remove, modify, or reorder demonstrations.
Format	$E_{10}-E_{11}$	O	Add or change output format, fields, labels, or schema.
Role	$E_{12}-E_{14}$	R	Add, remove, or modify persona/role framing.
Surface	$E_{17}-E_{20}$	layout	Reformat, reorder, clarify wording, or correct typos.

and cores. Perrone and Romano’s PEaC [11] treated prompts as YAML-serialized objects with explicit instruction, context, input, and output fields. Ronanki et al. [16] catalogued prompt smells from conflated components.

B. Operator Taxonomy

PROMPTVER defines 20 fine-grained edit operators E_1-E_{20} , each describing a single transformation between P and P' that touches one primary component. Table I summarizes the complete operator families; the replication package gives the full operator-level catalog and examples.

The families are grounded in prior empirical work: Instruction edits (E_1-E_3, E_{15}) influence task prioritization [1], [5]; Constraint edits (E_4-E_6) alter the feasible response space [19]; Example edits (E_7-E_9, E_{16}) affect in-context generalization [4]; Format edits ($E_{10}-E_{11}$) affect decoding stability [19]; Role edits ($E_{12}-E_{14}$) induce global behavioral shifts with high variance [8], [19]; Surface/restructuring edits ($E_{15}-E_{20}$) are form-level modifications consistent with dominant patterns in real workflows [6], [15]. Beyond our controlled study, an empirical analysis of prompt-evolution histories in open-source repositories confirms that practitioners evolve prompts predominantly through incremental additions, modifications, restructurings, and corrections; the same operations this taxonomy names [21]. The taxonomy is operator-centric: family does not predetermine label. A constraint addition (E_4) can be PATCH if it tightens cosmetic phrasing or MAJOR if it breaks a relied-upon behavior; the rule-guided procedure determines severity based on the components affected.

C. Rule-Guided Classification Procedure

Given (P, P') and the executor model identifier, PROMPTVER produces a label via sequential structural checks (Algorithm 1). Each step inspects a specific property and may commit to a label, escalate a candidate, or defer.

Implementation as a structured single-judge. The procedure is realized as a single LLM call whose system prompt encodes the decision rules and whose output is structured JSON containing the neutral diff, affected components, contract checks, committed label, and step-citing justification. Three independent runs with majority voting form the deployed classifier. The judge receives the prompt pair and the executor identifier, but never observes executor outputs; its decision is structural, not behavioral. Operationally, a prompt-touching

Algorithm 1 PROMPTVER Classification Procedure

Require: prompt pair (P, P') , executor model M

Ensure: SemVer label $\ell \in \{\text{PATCH}, \text{MINOR}, \text{MAJOR}\}$, triggering step

```
1: produce neutral diff over  $\langle R, D, Ctx, E, O \rangle$ 
2: identify task families  $t, t'$ 
3: if  $t \neq t'$  and the change is more than wording clarification then
4:   return (MAJOR, "task change")
5: end if
6: if required template variable or input contract changes then
7:   return (MAJOR, "input contract")
8: end if
9: if required downstream-visible output contract changes then
10:  return (MAJOR, "output contract")
11: end if
12: classify  $\Delta_{Ctx}$  as Style / Safety / Decision-rule / Hard-validity
13: if Decision-rule changes valid output distribution or Hard-
    validity changes parser assumptions then
14:  return (MAJOR, "constraint contract")
15: else if Style or Safety only then
16:   $cand \leftarrow \text{MINOR}$  (PATCH if redundant)
17: end if
18: if examples demonstrate different task or contradict directive
    then
19:  return (MAJOR, "example drift")
20: else if clean examples added/removed then
21:   $cand \leftarrow \max(cand, \text{MINOR})$  for classification; PATCH for
    clear summarization
22: end if
23: handle role/persona and surface edits
24: apply generation-specific calibration; verify final consistency
25: return ( $cand$ , triggering step)
```

commit flows through five artifacts: prompt pair, component diff, operator assignment, contract checks, and a SemVer label with an audit record.

III. METHODOLOGY

We evaluate PROMPTVER using a controlled benchmark. For each prompt edit, we execute the original and edited prompts and use the observed behavior to build reference labels. We release the edit catalog, judge prompts, execution outputs, labeling scripts, and analysis code in the replication package.

A. Edit Catalog and Stratification

The catalog contains 141 prompt-edit pairs across three task families: TREC (6-way topic classification, 48 edits), DBPedia (14-way entity classification, 48 edits), and XSum (single-sentence summarization, 45 edits). For evaluation, edits are stratified into seven *evaluation groups* probing different operator clusters: **A** surface and exemplar density, **B** constraint addition/removal, **C** persona, **D** format/schema, **E** example/role combinations, **F** Missing operators, and **G** adversarial consistency-check edits. This stratification is an evaluation device, not part of the framework definition.

B. Executor Models

We run every edit under four executor models: gemini-2.5-pro, gemini-2.5-flash, gemini-2.5-flash-lite,

and llama-3.1-8b-instant. The judge is a separate model (gpt-4o) not in the executor set. All runs use temperature 0, top- $p = 1$, max 512 tokens for classification and 256 for summarization. Each edit contributes one row per available executor; analyses cluster at the edit level.

C. Reference Labels from Execution

To evaluate PROMPTVER, we need reference labels for each prompt edit. These labels are not produced by PROMPTVER. They are produced from prompt execution.

For each edit, we run both the original prompt P and the edited prompt P' on the same inputs. We then compare their outputs. If the outputs change only within normal prompt sensitivity, we do not treat the edit as a meaningful behavioral change. Prior work shows that LLMs can change their outputs even when prompts are only rephrased without changing the task [4], [19], [25]. We therefore measure the output variation caused by meaning-preserving prompt rephrasings. This gives us a noise floor: changes below this level are treated as normal LLM sensitivity, while larger changes are treated as evidence of behavioral change.

The noise floor is only a calibration step. For each task and executor model, we create $k = 30$ meaning-preserving variants of the baseline prompt. These variants keep the same task, constraints, and output format. We run all variants on the same inputs and compare every pair of outputs. This gives $\binom{30}{2} = 435$ distances per input. We use the 95th percentile as the noise ceiling. In simple terms, this tells us how much output change can happen even when the prompt meaning is unchanged.

We remove unstable inputs before assigning labels. If an input already produces highly variable outputs under equivalent prompt variants, it is not useful for judging a real edit. Inputs with median distance above $\theta = 0.10$ are removed. The remaining inputs form the stable witness set $\mathcal{D}_s$.

We then measure the effect of each real edit on $\mathcal{D}_s$. For classification tasks, we compute the flip rate, i.e., how often the predicted class changes. For summarization tasks, we compute semantic drift using $1 - \text{BERTScore}_{F_1}$. Changes below the noise ceiling are treated as normal variation. Changes above the noise ceiling are treated as evidence of behavioral change.

Finally, we add explicit contract checks. Some edits are breaking even if the semantic distance is small. For example, changing the output format may preserve the meaning of the answer but still break a downstream parser. Therefore, if an edit changes the task, required input variables, output schema, parser-visible structure, output language, required fields, or label space, we label it as MAJOR. The final reference label combines execution-based evidence with these contract checks.

D. Baselines

We compare PROMPTVER against two baselines: a naive predictor that establishes a lower bound, and a direct LLM-based judge that tests whether structured rules add value over raw model capability **Always-PATCH**. Predicts PATCH on

TABLE II
CONTROLLED-BENCHMARK RESULTS. ACCURACY ALONE IS INSUFFICIENT FOR VERSIONING; DANGEROUS MISSES MEASURE BREAKING EDITS PREDICTED AS PATCH.

Method / Scope	Acc.	M-F1	Dang.-miss	Rows
Always-PATCH (global)	0.670	0.267	1.000	436
Simple-LLM (global)	0.673	0.586	0.0485	436
PROMPTVER – Global	0.718	0.549	0.0483	436
PROMPTVER – Classification	0.730	0.606	0.0667	322
PROMPTVER – XSum	0.684	0.525	0.000	114

every input; tests whether accuracy is inflated by class distribution and quantifies the dangerous-miss cost of a maximally optimistic predictor.

Simple-LLM SemVer judge. Uses the same judge model and decoding protocol as PROMPTVER, but receives only the SemVer class definitions and three short examples: no rule-guided procedure, component decomposition, contract checks, or generation calibration. Tests whether a strong LLM can infer prompt-edit severity directly from the prompt pair.

Both LLM-based systems see the same prompt pairs and executor identifiers and are evaluated against the same empirical reference.

E. Metrics

Exact accuracy, macro-F1, Kendall’s τ_b , per-class precision/recall/F1, MAJOR recall $P(\hat{\ell}=\text{MAJOR} \mid \ell_{\text{emp}}=\text{MAJOR})$ as the safety-critical catch rate, dangerous-miss rate $P(\hat{\ell}=\text{PATCH} \mid \ell_{\text{emp}}=\text{MAJOR})$, and over-conservative rate $P(\hat{\ell}=\text{MAJOR} \mid \ell_{\text{emp}}=\text{PATCH})$. We also check whether the explanation is consistent with the final label. For systems that produce a rationale, we count cases where the rationale mentions a breaking change but the system still predicts a non-MAJOR label. We call this the *faithfulness* error rate. This follows prior work showing that LLM explanations can be unfaithful to their final answers [22]. For paired comparisons, we use only the rows that are available for both LLM-based systems and for the reference labels.

IV. RESULTS

A. Headline Results and Trivial Baselines

After execution checks, noise-floor calibration, stable-witness filtering, and hard-violation-aware labeling, the retained benchmark contains 436 valid edit-executor rows. Table II reports accuracy and the safety diagnostics most relevant to versioning. Dangerous misses are especially important because they measure breaking edits predicted as PATCH.

The always-PATCH predictor reaches 0.670 accuracy purely because the PATCH class is the majority, but its macro-F1 collapses to 0.267 and its dangerous-miss rate is 1.000—it labels *every* breaking edit as a non-event. This is the worst-case operational profile: silent regressions on every MAJOR edit, including the contract failures hard-violation labels were designed to catch. PROMPTVER’s 4.8-point accuracy gain over this floor is therefore preliminary rather than decisive; the more important question is whether the system reduces dangerous misses while preserving an auditable safe-change signal.

TABLE III
COMPARISON WITH THE SIMPLE-LLM SEMVER BASELINE. MAJOR RECALL IS SAFETY-CRITICAL BECAUSE MISSED MAJOR EDITS CAN HIDE BREAKING CHANGES.

Method	Global Acc.	Class. Acc.	XSum Acc.	Class. MAJOR rec.	Trace
Always-PATCH	0.670	0.659	0.702	0.000	No
Simple-LLM SemVer judge	0.673	0.662	0.719	0.795	No
PROMPTVER	0.718	0.744	0.684	0.918	Yes

TABLE IV
TRACE EXAMPLE ON EDIT G-X5/DBPEDIA. REFERENCE LABEL: MAJOR.

Simple-LLM	MINOR (3/3). “adds intermediate output <i>before the final JSON object</i> ...backward-compatible.” Names the breaking property; dismisses it.
PROMPTVER	MAJOR (3/3). “ <i>Step 2 (Output Contract Check) is decisive: reasoning text before the JSON object alters the output contract.</i> ” Label bound to Alg. 1, line 9.

B. Comparison with a Simple LLM Baseline

Table III compares PROMPTVER with the Simple-LLM SemVer baseline.

The Simple-LLM baseline is competitive in global accuracy, but it misses more breaking edits. On classification tasks, PROMPTVER improves accuracy from 0.662 to 0.744 and MAJOR recall from 0.795 to 0.918. This matters because a missed MAJOR edit may hide a change that breaks application code.

The missed MAJOR cases follow a common pattern. The Simple-LLM often treats output-contract changes as backward-compatible, such as adding text before a JSON object, adding numbered reasoning before the answer, or changing the allowed label space. These edits may preserve the task intent, but they can still break code that expects the old output format.

On XSum, the Simple-LLM baseline performs better than PROMPTVER. This shows a limitation of the current rule-guided procedure: open-ended generation is harder to calibrate than classification.

We also check whether each explanation agrees with its final label. We call a case a *faithfulness failure* when the explanation mentions a breaking property, but the system still predicts a non-MAJOR label. For example, a rationale may say that the edit changes the output format, but the final label is still MINOR. This follows prior work showing that LLM explanations can be unfaithful to their final answers [22]. Among MAJOR reference cases, the Simple-LLM has 32 such failures, while PROMPTVER has 3. This suggests that structured traces make the decision easier to inspect than free-form rationales. Table IV contrasts the two outputs on a single edit. The Simple-LLM rationale names the breaking property (“*before the final JSON object*”) yet votes MINOR; PROMPTVER cites the firing step of Algorithm 1. Because PROMPTVER’s rationale references named steps and contract checks, a regex parser recovers a triggering check for 917 of 1,014 runs (90.4%), against 24.9% loose mentions and 0% step citations for the Simple-LLM. The output is a label plus a machine-extractable trace, not only a label plus prose. The main advantage of PROMPTVER is therefore not only the final

label. Its trace records the affected component, the contract check, and the rule that triggered the label.

C. Per-Task and Cross-Executor Observations

Classification tasks are easier to evaluate because changes are directly visible: labels can flip, the allowed label set can change, or the output can fail to parse. XSum is harder. A prompt edit may change style while keeping the same meaning, or it may change the output format in a way that matters to application code but is not well captured by BERTScore.

The stronger XSum result of the Simple-LLM baseline shows a limitation of the current PROMPTVER rules. They are better suited to classification than to open-ended generation. Most errors occur when the structural edit and the observed behavior disagree, especially for example, style, and cross-component edits in XSum.

V. DISCUSSION AND IMPLICATIONS

Findings. Our results show that prompt-edit severity can be estimated from structured prompt differences, but the problem is not solved. PROMPTVER reaches 0.718 global agreement and improves over the always-PATCH baseline. On classification tasks, it is better than the Simple-LLM baseline at catching MAJOR edits, improving MAJOR recall from 0.795 to 0.918. This matters because missed MAJOR edits can hide breaking changes.

The main errors of the Simple-LLM baseline occur on output-contract edits. It sometimes says an edit is backward-compatible even when the output format changes, such as adding text before JSON or changing the allowed label space. These edits may still break application code. The faithfulness analysis confirms this pattern: in 15.0% of true-MAJOR runs, the baseline explanation mentions a breaking property but still predicts a non-MAJOR label. This rate is 1.4% for PROMPTVER.

On summarization, the Simple-LLM baseline performs better. This shows a limitation of the current rules. Open-ended generation needs different calibration from classification. Overall, PROMPTVER should not be seen as a much stronger classifier. Its main value is the structured trace: it records the affected component, the triggered rule, and the reason behind the label.

Refinements and implications: Future work should simplify the rule-guided prompt, add rules specific to open-ended generation, and improve how contract violations are handled in the reference labels. PROMPTVER is not a behavioral oracle. It cannot prove that a prompt edit is safe. However, it can give developers an early and reproducible signal before deployment. In a CI pipeline, MAJOR edits could be flagged for careful review, MINOR edits could be marked as review-needed, and PATCH edits could be treated as lower risk.

VI. THREATS TO VALIDITY

Construct. Since prompts lack explicit machine-checkable contracts, ℓ_{emp} is an operational reference rather than absolute

ground truth. Our results measure agreement with this reference, not perfect semantic correctness.

Internal. The same team designed the edit catalog and PROMPTVER rules, which may bias the results. We mitigate this by releasing artifacts and comparing with a competitive Simple-LLM baseline.

Reference labels. Some labels use explicit failure checks, such as format changes, new required inputs, or parser-visible violations. These checks catch breaking edits missed by semantic metrics, but they partially overlap with PROMPTVER’s contract checks.

Faithfulness. Our faithfulness analysis relies on keyword matching in rationales. It detects obvious inconsistencies, but is only a proxy for explanation quality.

External. We evaluate only English classification and summarization tasks, with fixed models, context, and tests. Results may not transfer to extraction, tool use, RAG, multilingual prompts, or agentic systems.

VII. RELATED WORK

PROMPTVER sits at three intersections. *SemVer* was codified for communicating release impact [12], with empirical work documenting its value and imperfect adherence on machine-checkable APIs [3], [9], [14]. Prompts lack such a surface, motivating adaptation rather than direct application.

Prompts as software artifacts. Tafreshipour et al. [21] mined prompt-edit commits, Liang et al. [6] surveyed practitioner workflows and found ad-hoc version control, others proposed structured composition [8], [11], [16], and prompt-program frameworks expose typed surfaces at compile time [1], [5], [10], [13]. Closer to runtime, Shankar et al. [20] treat prompt evolution as a testing problem. Across this work, prompts are decomposed, surveyed, structured, and tested, but no thread assigns each edit a release-time severity label downstream consumers can act on.

LLM judges and prompt sensitivity. Prior work uses LLMs as judges [2], [7], [24] or quantifies sensitivity to semantically equivalent rephrasings [4], [19], the latter motivating our paraphrastic noise floor. PROMPTVER’s judge differs in that the artifact judged is a prompt-edit pair (not a model output) and the target is a structured three-class label tied to a versioning discipline.

VIII. CONCLUSION

Prompt edits are becoming part of software evolution, but their compatibility impact remains difficult to communicate. This paper introduced PROMPTVER, an early framework that assigns SemVer-style severity labels to prompt edits from structural changes in the prompt template.

Our results show that the problem is feasible but not solved. PROMPTVER improves over an always-PATCH baseline and better detects breaking edits in classification tasks than a direct LLM baseline. However, the LLM baseline remains competitive, especially for summarization.

The main value of PROMPTVER is therefore not accuracy alone, but its reproducible decision trace .

AI use. An LLM-based writing tool was used for language editing and clarity. All technical contributions, experimental design, analyses, and conclusions are the authors' own.

REFERENCES

- [1] L. Beurer-Kellner, M. Fischer, and M. Vechev, "Prompting Is Programming: A Query Language for Large Language Models," *Proc. ACM PLDI*, 2023.
- [2] C.-H. Chiang and H.-Y. Lee, "Can Large Language Models Be an Alternative to Human Evaluations?" *Proc. ACL*, 2023.
- [3] A. Decan and T. Mens, "What do package dependencies tell us about semantic versioning?" *IEEE TSE*, vol. 47, no. 6, pp. 1226–1240, 2021.
- [4] O. Honovich, U. Shaham, S. R. Bowman, and O. Levy, "Instruction Induction: From Few Examples to Natural Language Task Descriptions," *Proc. ACL*, 2023.
- [5] O. Khattab *et al.*, "DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines," *Proc. ICLR*, 2024.
- [6] J. T. Liang *et al.*, "Prompts Are Programs Too! Understanding How Developers Build Software Containing Prompts," *Proc. ACM FSE*, 2025.
- [7] Y. Liu *et al.*, "G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment," *Proc. EMNLP*, 2023.
- [8] S. Morales, R. Clarisó, and J. Cabot, "Impromptu: A Framework for Model-Driven Prompt Engineering," *Softw. Syst. Model.*, Springer, 2024.
- [9] S. Mostafa, R. Rodriguez, and X. Wang, "Experience Paper: A Study on Behavioral Backward Incompatibilities of Java Software Libraries," *Proc. ISSSTA*, 2017.
- [10] R. Willard *et al.*, "Outlines: Structured Text Generation," <https://github.com/dottxt-ai/outlines>, 2024.
- [11] G. Perrone and S. P. Romano, "PEaC: Prompt Engineering as Code – An Approach for Building Modular, Reusable, and Portable Prompts," *Proc. SEAA*, IEEE, 2024.
- [12] T. Preston-Werner, "Semantic Versioning 2.0.0," <https://semver.org/>, 2013.
- [13] Pydantic AI Contributors, "Pydantic AI: Agent Framework for Production Generative AI," <https://ai.pydantic.dev/>, 2024.
- [14] S. Raemaekers, A. van Deursen, and J. Visser, "Semantic versioning and impact of breaking changes in the Maven repository," *J. Syst. Softw.*, vol. 129, pp. 140–158, 2017.
- [15] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond Accuracy: Behavioral Testing of NLP Models with CheckList," *Proc. ACL*, 2020.
- [16] K. Ronanki, B. Cabrero-Daniel, and C. Berger, "Prompt Smells: An Omen for Undesirable Generative AI Outputs," *Proc. CAIN*, pp. 286–287, 2024.
- [17] E. Schluntz, "Engineering Prompts: Lessons from Production LLM Applications," Industry talk, 2024.
- [18] T. Schnabel and J. Neville, "Symbolic Prompt Program Search," *arXiv preprint arXiv:2404.02319*, 2024.
- [19] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr, "Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design," *Proc. ICLR*, 2024.
- [20] S. Shankar *et al.*, "SPADE: Synthesizing Data Quality Assertions for LLM Pipelines," *Proc. VLDB*, 2024.
- [21] M. Tafreshipour *et al.*, "Prompting in the Wild: An Empirical Study of Prompt Evolution in Software Repositories," *Proc. MSR*, 2025.
- [22] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting," *Proc. NeurIPS*, 2023.
- [23] J. Wang *et al.*, "Software Testing With Large Language Models: Survey, Landscape, and Vision," *IEEE TSE*, 2024.
- [24] L. Zheng *et al.*, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," *Proc. NeurIPS Datasets & Benchmarks*, 2023.
- [25] F. Errica, D. Sanvito, G. Siracusano, and R. Bifulco, "What Did I Do Wrong? Quantifying LLMs' Sensitivity and Consistency to Prompt Engineering," *Proc. NAACL*, 2025.